

The dashboard

The dashboard shows a SecureGate report in your web browser: the result, every finding, and how to fix each one. You can download the findings, or print the whole report or save it as a PDF. It is **read-only**: it reads the report file and never changes anything. It only ever shows **masked** values. It has a dark theme; printed pages are light.

Run it

After scanning the demo repo:

```
make demo
make scan-demo
make ui
```

`make ui` starts the dashboard and opens it in your browser at `http://127.0.0.1:5000`. To stop it, press **Ctrl+C** in the terminal.

You can also open it from SecureGate's menu: double-click `Start SecureGate.cmd` in the SecureGate folder on Windows, or run `make menu`. Choose 1 (or 2, with all four scanners) to scan the demo project and open the dashboard on the result, or 5 to open the dashboard on the last scan. While the dashboard is open, the menu waits: press Enter in its window to close the dashboard and go back to the menu.

The dashboard also shows the merge gate's reports from GitHub. `make ci-report` downloads the `findings.json` of the newest `secret-gate` run, checks it like any other report, saves it as `findings-ci.json` and opens it. In the menu, choose 9: after a demo pull request's check, Enter opens its report, and 7 opens the newest one.

To show another report, run this in the SecureGate folder:

```
.venv\Scripts\securegate ui --report findings.json --open
```

- `--report`: the report to show. The default is `findings.json`.
- `--port`: the port number. The default is 5000. A dashboard never shares its port: if 5000 is busy, for example because a dashboard is already open in another window, it stops and says so. Use the open one, or choose another port, for example `--port 5050`.
- `--open`: open the dashboard in your browser.
- `--debug`: print error details in the terminal. It is off by default.

The dashboard only answers on **127.0.0.1**, the address that always means "this computer": nobody else on your network can open it. It needs no internet connection: it loads nothing from other websites and runs no scripts.

If the report is missing or broken, the dashboard says so and shows the exact command that creates one. Run it, then reload the page. The report is read again every time a page opens, so a new scan appears after a reload.

The pages

Overview (/). At the top, the result in words: **BLOCKED** with the number of blocked findings, or **PASS**, and the exit code. Below it, four tiles: all findings, blocked, warnings and ignored; select a tile to see those findings. **Fix these first** lists the blocked findings, the most severe first, each with its masked value, file and line. One bar per severity shows how many findings have it. "About this scan" says when and where the scan ran, what was scanned (for example the whole Git history), the Gitleaks version and the policy file.

Findings (/findings). A table with the decision, severity, rule, file and line, masked value and reason. Blocked findings come first. The buttons above the table show one decision at a time; they change the address, for example to `/findings?decision=block`, so a filtered view can be bookmarked or shared. Select a row to open its details.

Finding detail (/findings/<id>). **How to fix**, in numbered steps, next to the main facts at a glance (decision, severity, policy rule, confidence, commit and author), then every field of the finding:

- for a blocked key: revoke it at the provider, create a new one, store it in a secret manager, then remove it from the code;
- for a warning or an ignored finding: why it was not blocked, and what to do if it turns out to be real.

The fields include **Found by** (every scanner that found it), **Live check** (whether TruffleHog confirmed with the provider that the key works) and **Policy rule** (the numbered rule that decided, such as `rule 8: provider-keys`).

If the same secret was found in several places, every place is listed. An id that is not in the report shows a "Not found" page.

Example: `/findings/dcc7b5bbfe3c` is the Stripe key from the demo's deleted script. It shows `sk_l****562d`, a BLOCK badge and four steps, starting with "Revoke it at the provider. In the Stripe Dashboard, open Developers > API keys and roll this key".

Report (/report). The whole report on one page: the result, the scan's details, the totals, and every finding with how to fix it. It is made for printing: see below.

Download the findings

The buttons at the top of the Overview and Findings pages download what SecureGate found:

Button	File	What it is
CSV	<code>findings-demo.csv</code>	One row per finding, for Excel or another spreadsheet: decision, severity, rule, file, line, masked value, policy rule, reason, fix, commit, author, date, confidence, entropy, detector, id and fingerprint.
JSON	<code>findings-demo.json</code>	The same findings as a SecureGate report, for other programs. The dashboard and <code>securegate summary</code> can open it too.
Markdown summary	<code>findings-demo-summary.md</code>	The report the merge gate shows on GitHub and posts on the pull request: the verdict, the findings table, a checklist to rotate every blocked key, and why each warning was not blocked. Paste it into a ticket or a pull request.
Printable report		Opens the Report page.

On the Findings page, CSV and JSON download only the findings shown: with the filter set to Block, you get `findings-demo-block.csv` with the blocked findings only. The file names come from the report's name.

Every download holds masked values only: the files are made from the same checked report that the pages show, so a report with an unmasked value is never downloaded. In the CSV file, a value that starts with `=`, `+`, `-` or `@`, such as the masked private key `-----*****-----`, gets an apostrophe in front, `'-----*****-----`, so that a spreadsheet shows it as text and never runs it as a formula.

Print it, or save it as a PDF

Open the **Report** page and press **Ctrl+P**. Choose your printer, or **Save as PDF** to get a PDF file. Printed pages are light: white paper, dark text, and the same coloured badges. The menus and buttons are left out, and a finding is never split across two pages when it fits on one. The other pages print the same way.

Change colours, fonts and spacing

Every design value lives in one file: `src/securegate/ui/static/css/tokens.css`. Change a value there and reload the page. The first block holds the dark theme for the screen; the block starting `@media print` holds the light colours for paper and PDFs.

To change	Edit these tokens
page, card, border and text colours	<code>--color-neutral-...</code>
the accent: links, bars, the active tab	<code>--color-brand-...</code>
the BLOCK, WARN and IGNORE badges	<code>--color-block-...</code> , <code>--color-warn-...</code> , <code>--color-ignore-...</code>
the PASS result	<code>--color-pass-...</code>
the coloured dots of the severities	<code>--color-severity-...</code>
fonts and text sizes	<code>--font-...</code> and <code>--line-height-...</code>
spacing	<code>--spacing-...</code>
rounded corners	<code>--border-radius-...</code>
the colours of printed pages and PDFs	the block starting <code>@media print</code>
text size and page width on big screens and projectors	the block starting <code>@media (min-width: 1600px)</code>

The names follow Fluent 2, so a Figma token such as `colorNeutralBackground1` becomes `--color-neutral-background-1`.

Two tests protect the design:

- `app.css`, the layout file, may only use these tokens. A test fails if a colour, size or spacing value is written anywhere else.
- Text colours must meet WCAG AA contrast (at least 4.5 to 1) against their backgrounds, on screen and on paper; the bars and coloured dots need at least 3 to 1. After changing a colour, run `make check` to see whether it still passes.

A decision is never shown by colour alone: it always has a BLOCK, WARN or IGNORE badge. A severity always has its name next to its dot.

A sample report for the designers

```
make sample-report
```

This writes `sample_findings.json` with about 20 findings. They come from a real scan of a demo repo that holds the DemoPay app twice: once as usual, and once more under `billing/`. It needs Git and Gitleaks. To look at it:

```
.venv\Scripts\securegate ui --report sample_findings.json --open
```